



Linux & Commands — Complete Notes

The OS of servers · Filesystem, permissions, processes & the command line



Topic Linux

Runs Most servers

Interface

Command line

DevOps

Essential

A reader-friendly, all-in-one reference for Linux and its commands.



Table of Contents

1. What is Linux
2. Filesystem Hierarchy
3. Navigation & Files
4. Viewing & Searching Files
5. Permissions & Ownership
6. Users & Groups
7. Processes & Jobs
8. Package Management
9. Networking Commands
10. Disk & System Info
11. Text Processing & Pipes
12. Services (systemd)
13. Archives & Compression
14. Command Cheat-Sheet
15. Quick Mental Model
16. Common Interview Questions

1. 🐧 What is Linux

Linux is an **open-source, Unix-like operating system kernel** that, with GNU tools, powers most **servers, cloud instances, containers, and Android**. You interact with it mainly through the **shell** (command line). Distributions (Ubuntu, Debian, RHEL, Amazon Linux, Alpine...) bundle the kernel with tools and a package manager.

💡 **Mental model:** in Linux **"everything is a file"** — devices, processes, and configs are accessed as files/paths. You compose small single-purpose commands with **pipes** to do powerful things. Mastering the CLI is the core DevOps skill.

Common uses: servers, cloud (EC2), containers (Docker base images), CI runners, embedded, networking gear.

2. 📁 Filesystem Hierarchy

One **single tree** rooted at `/` (no drive letters):

Path	Holds
<code>/</code>	Root of everything
<code>/home</code>	User home directories (<code>/home/alice</code>)
<code>/etc</code>	System configuration files
<code>/var</code>	Variable data — logs (<code>/var/log</code>), spool, caches
<code>/tmp</code>	Temporary files (cleared on reboot)
<code>/bin</code> , <code>/usr/bin</code>	Executables / commands
<code>/opt</code>	Optional / third-party software
<code>/dev</code>	Device files
<code>/proc</code> , <code>/sys</code>	Kernel/process info (virtual)
<code>/root</code>	The root user's home
<code>/mnt</code> , <code>/media</code>	Mount points for drives

Paths are **absolute** (`/etc/hosts` , from root) or **relative** (`./script` , from the current dir). `~` = your home, `.` = here, `..` = parent.

3. 📁 Navigation & Files

```
pwd                # print working directory
ls -lah            # list: long, all (hidden), human sizes
cd /var/log        # change dir  (cd ~ = home, cd - = previous)
tree               # directory tree

mkdir -p a/b/c      # make dirs (and parents)
touch file.txt      # create empty file / update timestamp
cp -r src/ dst/     # copy (recursive for dirs)
mv old new         # move / rename
rm -rf dir/         # remove recursively + force ⚠️ irreversible
ln -s target link   # symbolic link

find . -name "*.log" -mtime +7    # find logs older than 7 days
find . -type f -size +100M        # files over 100MB
```

⚠️ `rm -rf` has no undo and no trash — double-check the path before running, especially with `sudo` .

4. 👁️ Viewing & Searching Files

```
cat file           # print whole file
less file          # scroll (q to quit, / to search)
head -n 20 file    # first 20 lines
tail -n 50 file    # last 50 lines
tail -f /var/log/syslog # follow live (great for logs)
wc -l file         # count lines

grep "ERROR" app.log      # find matching lines
grep -ri "timeout" /etc   # recursive, case-insensitive
grep -n "TODO" *.py       # show line numbers
nano file                # simple editor (vim/vi = powerful, modal)
```

5. 🔒 Permissions & Ownership

Each file has permissions for **user (owner)** / **group** / **others**, each with **read (r=4)**, **write (w=2)**, **execute (x=1)**.

```
-rwxr-xr--  1 alice devs  ...
|_|_|_|_|
| u  g  o      u=rwx(7) g=r-x(5) o=r--(4) →  chmod 754
type
```

```
chmod 755 script.sh # rwxr-xr-x (numeric)
chmod +x script.sh  # add execute for all (symbolic)
chmod u+w,o-r file   # user +write, others -read
chown alice:devs file # change owner:group
chown -R alice dir/  # recursive
sudo command         # run as root (superuser)
umask                # default permission mask
```

Number	Perms	Means
7	rwx	read+write+execute
5	r-x	read+execute
4	r--	read only
6	rw-	read+write

6. 👤 Users & Groups

```
whoami          # current user
id              # uid, gid, groups
who / w         # who's logged in
sudo useradd -m bob # create user (with home)
sudo passwd bob  # set password
sudo usermod -aG sudo bob # add bob to 'sudo' group
sudo userdel -r bob # delete user + home
su - bob        # switch user
groups          # my groups
```

- **root** (uid 0) is the superuser; prefer **sudo** for one-off privileged commands over logging in as root.
- User info in `/etc/passwd` ; groups in `/etc/group` ; hashed passwords in `/etc/shadow` .

7. ⚙️ Processes & Jobs

```
ps aux          # all processes (snapshot)
ps aux | grep nginx # find a process
top             # live process monitor (htop = nicer)
kill 1234       # send SIGTERM to PID 1234 (graceful)
kill -9 1234    # SIGKILL (force) ⚠️ no cleanup
pkill nginx     # kill by name
killall firefox

sleep 60 &      # run in background (&)
jobs            # background jobs in this shell
fg %1          # bring job 1 to foreground
bg %1          # resume job 1 in background
nohup ./long.sh & # keep running after logout
Ctrl+C         # interrupt  Ctrl+Z = suspend
```

- **Signals:** **SIGTERM** (15, graceful, default) vs **SIGKILL** (9, force). Try TERM first.
- Each process has a **PID**; `$$` is the shell's PID, `$_` the last background PID.

8. 📦 Package Management

Family	Install	Update	Search	Remove
Debian/Ubuntu (<code>apt</code>)	<code>sudo apt install nginx</code>	<code>sudo apt update && sudo apt upgrade</code>	<code>apt search x</code>	<code>sudo apt remove nginx</code>
RHEL/Amazon/Fedora (<code>dnf</code> / <code>yum</code>)	<code>sudo dnf install nginx</code>	<code>sudo dnf upgrade</code>	<code>dnf search x</code>	<code>sudo dnf remove nginx</code>
Alpine (<code>apk</code>)	<code>apk add nginx</code>	<code>apk update && apk upgrade</code>	<code>apk search x</code>	<code>apk del nginx</code>

```
sudo apt update          # refresh package index (do first)
sudo apt install -y htop  # -y = assume yes
dpkg -l | grep nginx     # list installed (Debian)
which python3            # path of a command
```

9. Networking Commands

```
ip addr          # interfaces & IPs   (modern; old: ifconfig)
ip route         # routing table
ping -c4 google.com # connectivity test
curl -I https://x.com      # HTTP headers   (wget = download)
curl -s url | jq .        # fetch + parse JSON
ss -tulnp        # listening ports/sockets (modern; old: netstat)
dig example.com   # DNS lookup   (nslookup = alt)
ssh user@host     # remote shell
ssh -i key.pem ec2-user@1.2.3.4
scp file user@host:/path # secure copy
rsync -avz src/ host:/dst/ # efficient sync
nc -zv host 443      # test if a port is open
```

10. Disk & System Info

```
df -h          # disk space per filesystem (human)
du -sh *       # size of each item here
du -sh /var/log # total size of a dir
free -h        # memory usage
lsblk          # block devices / disks
mount / umount # mount or unmount a filesystem
uname -a       # kernel & arch
uptime        # load average + how long up
cat /etc/os-release # which distro/version
nproc         # number of CPUs
date          # current date/time
lsof -i :8080  # what's using port 8080
```

11. Text Processing & Pipes

The Unix superpower: **pipe** (`|`) the output of one command into the next.

```
cmd1 | cmd2          # pipe stdout → stdin
grep ERROR app.log | wc -l    # count error lines
cat access.log | cut -d' ' -f1 | sort | uniq -c | sort -nr | head
# ^read    ^col 1    ^sort ^count uniq ^by count ^top
```

Tool	Does
<code>grep</code>	Filter lines by pattern
<code>sed</code>	Stream edit (<code>sed 's/old/new/g'</code>)
<code>awk</code>	Field/column processing (<code>awk '{print \$1}'</code>)
<code>cut</code>	Extract columns (<code>cut -d, -f2</code>)
<code>sort / uniq</code>	Sort / dedupe (<code>sort uniq -c</code>)
<code>wc</code>	Count lines/words/bytes (<code>wc -l</code>)
<code>tr</code>	Translate/delete chars
<code>xargs</code>	Turn input into command args
<code>tee</code>	Write to a file and stdout

Redirection: `>` overwrite, `>>` append, `2>` stderr, `2>&1` merge, `&>/dev/null` discard, `<` read from file.

12. Services (systemd)

Most modern distros manage services with **systemd**:

```

sudo systemctl start nginx      # start now
sudo systemctl stop nginx
sudo systemctl restart nginx
sudo systemctl status nginx     # state + recent logs
sudo systemctl enable nginx     # start on boot
sudo systemctl disable nginx
systemctl list-units --type=service # all services

journalctl -u nginx -f          # follow a service's logs
journalctl -xe                  # recent errors
journalctl --since "1 hour ago"

```

`enable` = start at boot; `start` = start right now. Use `status + journalctl -u <svc>` to debug a failing service.

13. 🗄 Archives & Compression

```
tar -czf backup.tar.gz dir/      # create gzip archive (c=create z=gzip f=file)
tar -xzf backup.tar.gz           # extract (x=extract)
tar -tzf backup.tar.gz           # list contents (t=list)
gzip file # → file.gz            gunzip file.gz
zip -r out.zip dir/              # zip / unzip out.zip
```

Mnemonic for tar: **"Create Ze File"** (`-czf`) and **eXtract Ze File** (`-xzf`).

14. 📋 Command Cheat-Sheet

Task	Command
Where am I / list	<code>pwd</code> · <code>ls -lah</code>
Move / copy / delete	<code>mv</code> · <code>cp -r</code> · <code>rm -rf</code>
Find files	<code>find . -name "*.log"</code>
Search text	<code>grep -rni "text" .</code>
Follow a log	<code>tail -f /var/log/...</code>
Permissions	<code>chmod 755</code> · <code>chown user:grp</code>
Run as root	<code>sudo <cmd></code>
Processes	<code>ps aux</code> · <code>top</code> · <code>kill -9 PID</code>
Install package	<code>sudo apt install x</code>
Disk / memory	<code>df -h</code> · <code>du -sh *</code> · <code>free -h</code>
Ports listening	<code>ss -tulpn</code>
Service control	<code>systemctl restart x</code> · <code>journalctl -u x</code>
Remote / copy	<code>ssh user@host</code> · <code>scp f host:/p</code>
Archive	<code>tar -czf a.tgz dir/</code> · <code>tar -xzf a.tgz</code>
Command help	<code>man <cmd></code> · <code><cmd> --help</code>

15. 🧠 Quick Mental Model

- Linux = the OS of servers/cloud/containers, driven from the **shell**; "everything is a file."

- One tree from `/` : configs in `/etc` , logs in `/var/log` , homes in `/home` .
- Permissions = `rwx` for `user/group/other` → `chmod` (755) + `chown` ; `sudo` for root tasks.
- Manage processes with `ps` / `top` / `kill` ; jobs with `&` / `jobs` / `fg` / `nohup` .
- **Pipe** (`|`) small commands together; `grep/sed/awk/sort/uniq` for text; redirect with `>` / `>>` / `2>&1` .
- Install via the distro's package manager (`apt/dnf/apk`); control services with `systemctl` + `journalctl`.
- Networking: `ip/ss/curl/ssh/scp`; disk/mem: `df/du/free`; archives: `tar -czf / -xzf`.
- When stuck: `man <cmd>` or `<cmd> --help` .

16. ? Common Interview Questions

Rapid-fire questions interviewers ask about Linux:

- **Q: What does `chmod 755` mean?** — `rwx` for owner, `r-x` for group, `r-x` for others (7=`rwx`, 5=`r-x`). Common for executables/scripts.
- **Q: How do you find which process uses a port?** — `ss -tulpn | grep :8080` or `lsof -i :8080` .
- **Q: `kill` vs `kill -9` ?** — `kill` sends `SIGTERM` (graceful, lets the process clean up); `kill -9` sends `SIGKILL` (forced, no cleanup) — use only if `TERM` fails.
- **Q: How do you view live logs?** — `tail -f /var/log/...` or `journalctl -u <service> -f` .
- **Q: Difference between `>` and `>>` ?** — `>` overwrites a file; `>>` appends to it.
- **Q: How do you check disk and memory usage?** — `df -h` (disk per filesystem), `du -sh *` (per item), `free -h` (memory).
- **Q: What's the difference between absolute and relative paths?** — Absolute starts from `/` ; relative is from the current directory (`.` / `..`).
- **Q: How do you run a command as another/super user?** — `sudo <cmd>` for root; `su - user` to switch users.
- **Q: `systemctl start` vs `enable` ?** — `start` runs it now; `enable` makes it start on boot.
- **Q: How do you search recursively for text?** — `grep -rn "pattern" /path` .
- **Q: What does "everything is a file" mean?** — Devices, processes, and many kernel interfaces are exposed as file paths (e.g. `/dev` , `/proc`), so the same tools work on them.
- **Q: How do you make a script executable and run it?** — `chmod +x script.sh` then `./script.sh` .
- **Q: How do you keep a process running after logout?** — `nohup ./cmd &` , or use `tmux` / `screen` , or a `systemd` service.

 Notes compiled as a quick reference for Linux & its commands.